Emergent State Machines: A Deterministic Architecture for Situational Reasoning in Complex Systems

Richard Hines
Independent Researcher
Tacoma, Washington, USA
richardhinestacoma@gmail.com

Abstract

This paper presents the mathematical foundations of the Emergent State Machine (ESM), a deterministic architecture for interpretable situational reasoning in complex systems. The central organizing unit of the architecture is the *turn*: a complete reasoning frame in which observations from the environment are transformed into signals, assembled into an interpretable state vector, evaluated relative to policy-defined dimensions, and mapped deterministically to an action. By structuring system behavior as a sequence of turns, the ESM makes reasoning processes inspectable, replayable, and auditable.

Within each turn, raw observations are converted into direct and derived signals defined by domain experts. These signals form an explicit state vector representing the system's current situation. A projection mechanism then expresses that state in policy-relevant coordinates, enabling deterministic policy selection without collapsing reasoning into latent or opaque representations. Repeated turns generate trajectories through interpretable state space, allowing policies to guide the system toward desirable stable regions and away from undesirable ones.

Unlike architectures that entangle probabilistic interpretation with consequential action, the ESM preserves a structural boundary between state representation and authoritative policy selection. This separation supports traceability, failure localization, and controlled system evolution through versioned revision of signals, projections, and policies. The resulting framework provides a mathematically structured approach to decision systems whose reasoning must remain visible, reproducible, and governable across domains including clinical monitoring, learning systems, and operational control.

github.com/emergent-state-machine/esm-spec

Contribution Statement. This paper makes four principal contributions. First, it defines the *turn* as the atomic reasoning object of an Emergent State Machine, providing a formal unit for inspection, replay, and audit. Second, it presents a deterministic state-space formulation in

which observations are transformed into signals, state vectors, policy projections, and actions through an explicit reasoning chain. Third, it shows how repeated turns generate trajectories through interpretable state space, enabling policy to guide system evolution toward desirable stable regions. Fourth, it formalizes *Instrumented Deterministic Evolution* as a regime in which behavioral change occurs only through explicit revision of versioned architectural artifacts rather than through hidden parameter drift.

1 Introduction

Many contemporary decision systems operate as opaque pipelines in which complex transformations produce outputs without exposing the intermediate reasoning steps that led to those outputs. In domains such as clinical care, education, and operational control, this opacity creates a practical problem: professionals remain responsible for consequential decisions, yet the computational systems supporting those decisions often do not provide an inspectable account of how those decisions were reached. Threshold-triggered alerting systems partially address this problem by surfacing selected measurements, but they typically expose signals rather than situations. Human operators must still reconstruct the situational meaning of those signals for themselves.

The Emergent State Machine (ESM) addresses this limitation by structuring automated reasoning into discrete, inspectable units called *turns*. A turn is the fundamental reasoning frame of the system. During a turn, observations are collected from the environment, transformed into direct and derived signals, assembled into an interpretable state vector, evaluated through policy-relevant projections, and mapped deterministically to an action. The turn therefore serves as the minimal object required to inspect, replay, reproduce, and audit system reasoning.

This framing is important for both theory and practice. Theoretically, it allows the ESM to be modeled as a dynamical system evolving through an interpretable state space. Practically, it gives system designers and domain experts a concrete artifact that can be logged, examined, and revised. Rather than treating reasoning as an invisible process inside a model, the ESM makes each reasoning step explicit. Each turn packages the information available to the system, the transformations applied to that information, the policy dimensions activated, and the action selected. A sequence of turns therefore provides both a trajectory through state space and a structured record of how the system's reasoning unfolded.

The architecture developed here differs from systems that entangle probabilistic interpretation with authoritative action selection. In the ESM, observations, signals, state representation, policy projection, and policy action remain structurally separated. This separation preserves interpretability and makes system behavior traceable. The resulting reasoning chain can be inspected in both directions: from observations forward to action, and from action backward to the observations and rules that produced it. Because the architecture is deterministic at the

authoritative layer, system behavior is replayable under fixed versions of its governing artifacts.

This paper presents the mathematical foundations of that architecture. It first defines the turn as the atomic unit of reasoning, then formalizes the transformation from observations to signals, state vectors, and policy space. It next describes state evolution, trajectories, desirable regions, and stable states in interpretable state space. Finally, it shows how determinism, auditability, and replayability support a regime of *Instrumented Deterministic Evolution*, in which system behavior changes only through explicit revision of versioned architectural components.

- observations, representing measurements or events in the monitored environment
- signals, which are direct or derived quantities computed from observations
- state vectors, organizing signals into a structured representation of the current situation
- projection into policy space, evaluating the state relative to defined decision dimensions
- policy selection, determining the appropriate action for the identified situation
- state evolution, occurring as the system receives new observations over time

By making each stage explicit and inspectable, the ESM provides a transparent alternative to black-box decision systems. Each decision can be traced through a deterministic chain linking actions to rules, signals, and raw observations.

The architectural principles underlying the Emergent State Machine draw on ideas from control theory, dynamical systems, software architecture, and research on situation awareness and AI governance. The relationship between these traditions and the ESM architecture is discussed in Section 13.

2 The Turn as the Fundamental Unit of Reasoning

The Emergent State Machine organizes reasoning into discrete computational units called *turns*. A turn represents one complete cycle of observation, interpretation, policy evaluation, and action selection. Rather than treating system reasoning as a continuous and opaque transformation, the ESM treats each turn as an explicit object that can be inspected, replayed, and reproduced.

At time t, a turn may be represented abstractly as

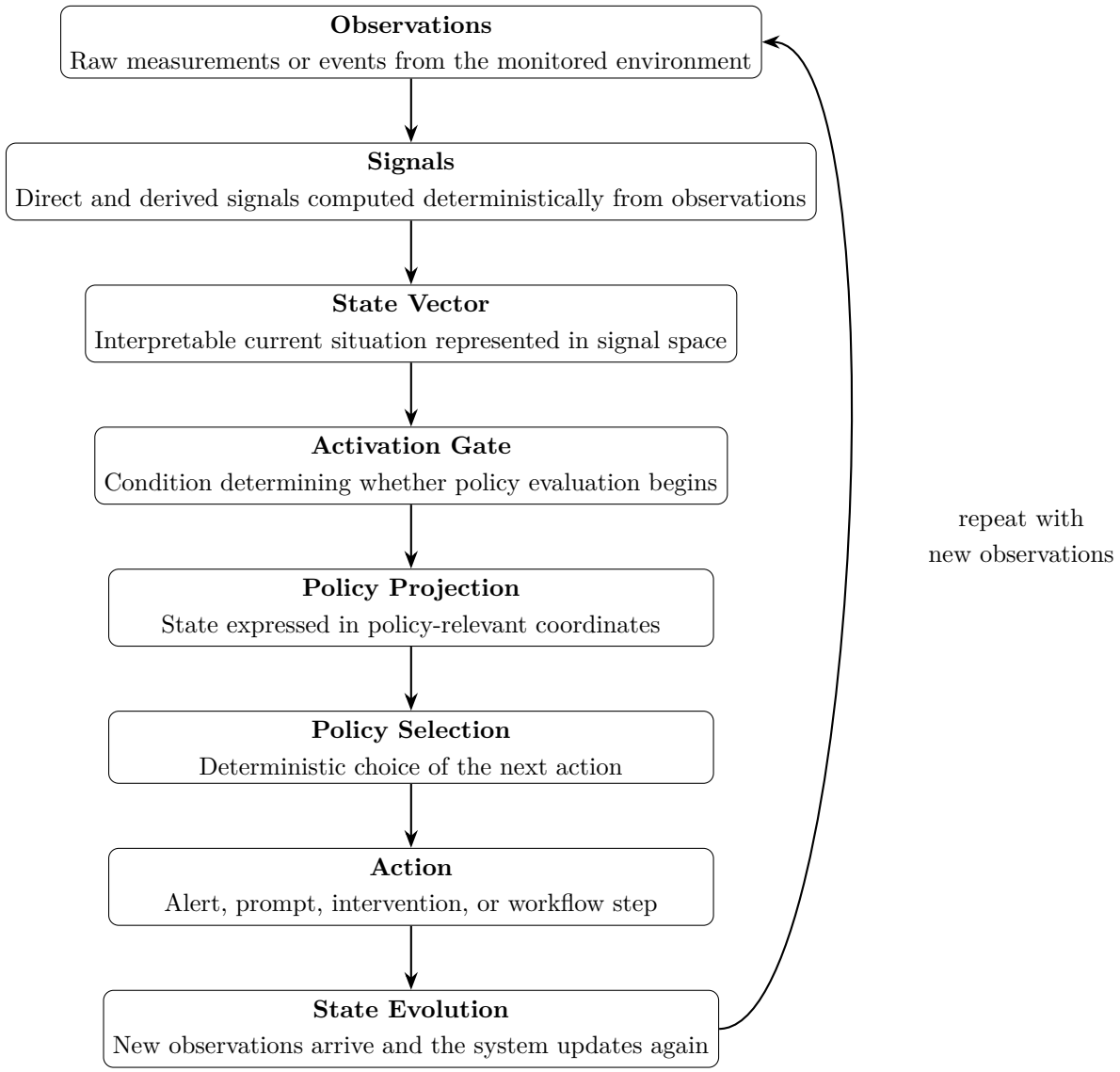
$$T_t = (o_t, S_t, x_t, g_t, y_t, a_t) \tag{1}$$

where

- o_t denotes the observations available at time t,

- S_t denotes the set of direct and derived signals computed from those observations,
- x_t denotes the interpretable state vector constructed from those signals,
- g_t denotes the activation gate state,
- y_t denotes the representation of the state in policy coordinates,
- a_t denotes the action selected by policy.

This formulation makes explicit that a turn is not merely a timestamped input or output. It is the full reasoning frame of the system at that moment. Each turn contains the information needed to reconstruct the system's situational interpretation and the policy logic that produced its action.



Trajectory through State Space: $x_0 \to x_1 \to x_2 \to x_3 \to \dots$

Control Objective: steer trajectories toward desirable or stable regions

Figure 1: Core reasoning loop of an Emergent State Machine. Each turn transforms observations into signals, constructs a state vector, projects the state into policy coordinates, and selects an action.

At initialization, before observations have been received, the system has no domain-specific state for the monitored case:

$$T_0 = (\varnothing, \varnothing, \varnothing, 0, \varnothing, \varnothing) \tag{2}$$

However, the system is not structurally empty. The signal definitions, gate conditions, policy dimensions, and policy rules already exist as versioned architectural artifacts. The first non-empty turn begins when observations arrive and the system constructs its first signal set and state vector.

A sequence of turns forms the temporal structure of ESM reasoning:

$$T_0 \to T_1 \to T_2 \to \dots \tag{3}$$

This sequence corresponds to the evolution of the monitored system through state space. Because each turn remains explicit and inspectable, the system's reasoning process can be analyzed at the level of individual turns or at the level of trajectories formed across many turns.

The concept of the turn plays two roles in the architecture. First, it is the atomic object of inspection and audit. Second, it is the temporal unit through which state trajectories are generated. In this sense, turns connect the operational behavior of the system to its mathematical formulation. What evolves over time is not an abstract hidden process, but a sequence of explicit turns whose contents remain available for replay and analysis.

3 Observations and Signals

Each turn of an Emergent State Machine begins with observations obtained from the environment being monitored. Observations represent measurements, events, or reported information describing the system at time t.

Observations are denoted

$$o_t \tag{4}$$

where o_t represents the collection of raw inputs available to the system during turn t. These inputs may originate from sensors, instruments, logs, or structured human reporting.

Observations themselves are not directly used for policy evaluation. Instead, they are transformed into *signals*, which provide structured, policy-relevant quantities derived from the available observations.

Signals fall into two categories: *direct signals* and *derived signals*.

3.1 Direct Signals

Direct signals correspond directly to measured or reported quantities within the observations. A direct signal is defined as

$$s_i^{direct} = m_i(o_t) \tag{5}$$

where m_i is a deterministic measurement function extracting the relevant value from the observations.

Examples of direct signals include quantities such as heart rate, respiratory rate, oxygen saturation, or a recorded student response in an educational setting.

Direct signals represent the immediate measurable properties of the system at the current turn.

3.2 Derived Signals

Derived signals represent deterministic computations defined over observations or other signals. These signals capture relationships, trends, or structural properties that are not directly observable but can be computed from available measurements.

A derived signal may be written as

$$s_j^{derived} = f_j(o_{t-k}, \dots, o_t) \tag{6}$$

or equivalently

$$s_j^{derived} = f_j(s_1, s_2, \dots) \tag{7}$$

where f_j is a deterministic function defined by system designers or domain experts.

Derived signals allow the system to represent domain-specific quantities such as ratios, differences, persistence conditions, or rates of change. For example, a common clinical derived signal is the shock index

$$shock_index = \frac{HR}{SBP}. \tag{8}$$

Derived signals may also incorporate temporal structure by referencing observations across multiple turns.

3.3 Signal Set

The complete set of signals available during turn t is denoted

$$S_t = \{s_1, s_2, \dots, s_n\}. \tag{9}$$

Signals provide the structured measurement interface between raw observations and the system's internal representation of the current situation. Because signals are explicitly defined and versioned, they remain interpretable and auditable across system revisions.

4 State Vector Construction

Signals derived during a turn are organized into a structured mathematical representation of the system's current condition called the state vector. Representing system behavior through explicit state vectors is standard practice in control and system identification theory [Ljung, 1999].

The practice of representing system conditions through structured feature measurements derived from observable inputs is common in statistical learning and signal-processing systems [Bishop, 2006, Raghavan and Kulkarni, 2014]. In the ESM architecture, these signals remain explicitly defined and inspectable rather than embedded within latent model parameters.

The state vector at time t is denoted

$$x_t \tag{10}$$

and constructed from the available signals as

$$x_t = \begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_n \end{bmatrix}. \tag{11}$$

Each component of the vector corresponds to a signal computed from the observations available during the current turn.

Unlike latent representations commonly used in many machine learning systems, the ESM state vector is explicit and interpretable. Each element corresponds to a signal whose definition, range, and computation are known. This design ensures that the system's representation of the current situation can be inspected, verified, and revised by domain experts.

The state vector therefore functions as the system's formal description of the situation being monitored at turn t. It captures the relevant measurable properties of the system in a structured mathematical form suitable for policy evaluation.

Because the state vector is constructed deterministically from signals, identical observations will always produce identical state vectors under fixed signal definitions. This property supports replayability and traceability of system reasoning.

Across successive turns, the state vector evolves as new observations arrive and new signals are computed. The resulting sequence

$$x_0, x_1, x_2, x_3, \ldots \tag{12}$$

forms a trajectory through the system's state space, representing the evolution of the monitored situation over time.

5 Activation Gates

Not every policy dimension or reasoning pathway is relevant for every possible system state. The Emergent State Machine therefore employs *activation gates* that determine whether particular reasoning structures become active for a given turn.

An activation gate evaluates conditions defined over the current state vector:

$$g_k = G_k(x_t) \tag{13}$$

where G_k is a deterministic function and

$$g_k \in \{0, 1\}. \tag{14}$$

If $g_k = 1$, the associated reasoning pathway or policy dimension becomes active. If $g_k = 0$, that pathway remains inactive for the current turn.

Activation gates serve two primary purposes.

First, they restrict reasoning to policy structures that are relevant to the current situation. Many systems contain policy rules designed to address specific conditions that occur only within particular regions of state space. Gates ensure that such rules activate only when the relevant structural conditions are present.

Second, gates support modular reasoning architectures. By activating only the relevant portions of the policy structure, the system avoids evaluating unnecessary decision logic while preserving transparency in how reasoning pathways are selected.

Activation gates may operate at multiple structural levels. Some gates control entry into high-level policy regimes, while others activate more specific reasoning pathways within those regimes. Regardless of their position in the reasoning structure, activation gates remain deterministic functions of the state vector.

Because gates are defined explicitly, the conditions under which reasoning pathways activate remain inspectable and auditable. When a particular policy dimension becomes active, the gate condition responsible for that activation can be traced directly to the signals that produced the corresponding state vector.

6 Projection into Policy Space

The state vector describes the current situation of the monitored system in terms of measurable signals. However, policy decisions are typically defined in terms of higher-level situational dimensions rather than individual signals. The Emergent State Machine therefore transforms the state vector into a representation expressed in policy-relevant coordinates.

This transformation is called *projection into policy space*.

Let P denote the policy projection matrix. The projection of the state vector is

$$y_t = Px_t \tag{15}$$

where

- x_t is the state vector constructed from signals,
- P is the policy projection operator,
- y_t is the representation of the state in policy coordinates.

Each component of y_t corresponds to the activation level of a policy-relevant dimension. These dimensions are defined by system designers or domain experts and represent structural properties of the situation being monitored.

For example, in a clinical monitoring system, projection dimensions might include degrees of respiratory instability, circulatory compromise, or metabolic disturbance. In an educational system, projection dimensions might represent claim strength, evidence relevance, or reasoning coherence.

Projection therefore does not predict the future. Instead, it illuminates the current situation by expressing the measured state in coordinates relevant to policy evaluation. Representing system behavior through transformations of state vectors into decision-relevant coordinates is consistent with state-space approaches in control and estimation theory [Kalman, 1960, Ljung, 1999].

Because projection is defined as an explicit transformation of the state vector, its behavior remains inspectable and reproducible. Changes to the projection operator correspond to explicit revisions of the system's interpretive structure rather than hidden modifications to latent representations.

7 Policy Selection and Action

Once the system's current state has been expressed in policy coordinates, the Emergent State Machine selects an action using deterministic policy rules.

Let π denote the policy function governing action selection. The action chosen at turn t is

$$a_t = \pi(y_t) \tag{16}$$

where

- y_t is the policy-space representation of the current state,
- π is the policy selection function,

- a_t is the action selected by the system.

Policy rules encode the decision logic that governs system behavior. These rules may include threshold conditions, escalation logic, persistence requirements, or other deterministic decision criteria defined by domain experts.

Because policy is deterministic, identical policy-space states will always produce identical actions under fixed policy definitions. This property ensures that system decisions remain reproducible and traceable.

The full reasoning chain for a turn can therefore be expressed as

$$a_t \leftarrow y_t \leftarrow x_t \leftarrow S_t \leftarrow o_t. \tag{17}$$

This chain allows any selected action to be traced backward through the policy projection, state vector, signals, and observations that produced it.

In the Emergent State Machine architecture, policy represents the authoritative decision mechanism of the system. While projection interprets the state of the monitored system, only policy determines which actions may be taken in response.

8 State Evolution and Trajectories

As new observations arrive, the Emergent State Machine constructs new turns and updates its representation of the monitored situation. Each turn therefore produces a new state vector derived from the latest observations.

Let o_{t+1} denote the observations available during the next turn. The system update can be written as

$$x_{t+1} = \Phi(x_t, o_{t+1}) \tag{18}$$

where Φ represents the deterministic transformation that incorporates new observations into the evolving state representation.

Successive state vectors form a trajectory through state space, a standard representation used in dynamical systems and control theory [Åström and Murray, 2010, Bertsekas, 1995]:

$$x_0, x_1, x_2, x_3, \ldots \tag{19}$$

This trajectory represents the temporal evolution of the monitored situation.

Because each state vector is constructed from explicitly defined signals, the system's trajectory through state space remains interpretable. Changes in system behavior can therefore be analyzed by examining how individual signals and policy dimensions evolve across successive turns.

Temporal relationships between observations may also be incorporated into derived signals. For example, signals may capture persistence conditions, trends, or rates of change across multiple turns. These temporal signals allow the system to detect structural patterns that emerge over time rather than relying solely on instantaneous measurements.

The sequence of turns

$$T_0 \to T_1 \to T_2 \to \dots \tag{20}$$

therefore corresponds to a trajectory through the system's interpretable state space.

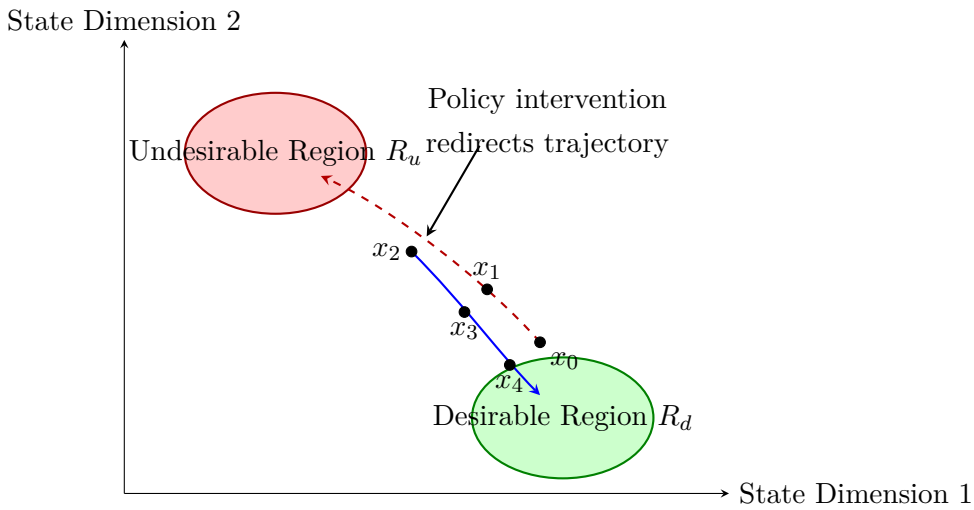


Figure 2: Example trajectory through ESM state space. System states evolve across successive turns, forming trajectories that may converge toward desirable stable regions or diverge toward undesirable conditions.

Trajectories through state space may converge toward desirable regions, as illustrated in Figure 2. Policy rules operate upon this evolving representation to guide the system toward desirable conditions and away from undesirable ones.

9 Desirable Regions and Stable States

Not all regions of state space correspond to acceptable or stable system conditions. Certain configurations of signals represent preferred operational states, while others indicate instability or risk. The Emergent State Machine therefore distinguishes between desirable and undesirable regions of state space.

Let $\mathcal{X}$ denote the space of all possible state vectors. Within this space, domain experts define a subset

$$\mathcal{X}_D \subset \mathcal{X} \tag{21}$$

representing desirable or stable system states.

For example, in a clinical monitoring system, the desirable region may correspond to combinations of respiratory rate, oxygen saturation, and blood pressure within physiologically stable ranges. In an educational system, desirable regions may correspond to combinations of signals indicating coherent reasoning and evidence alignment.

As the system evolves through successive turns, the sequence of state vectors

$$x_0, x_1, x_2, \ldots \tag{22}$$

forms a trajectory through state space. This trajectory may move toward or away from the desirable region $\mathcal{X}_D$.

Policy rules operate on projected state representations to influence these trajectories. When the system moves toward undesirable regions of state space, policy may trigger actions designed to restore the system to stable conditions.

In dynamical systems terminology, desirable regions may behave as *stable attractors* [Strogatz, 2015].. Interventions guided by policy act to steer the trajectory of the system back toward these regions when deviations occur.

Not all regions of state space correspond to acceptable or stable system conditions. Certain configurations of signals represent preferred operational states, while others indicate instability or risk. The Emergent State Machine therefore distinguishes between desirable and undesirable regions of state space.

Let $\mathcal{X}$ denote the space of all possible state vectors. Within this space, domain experts define a subset

$$\mathcal{X}_D \subset \mathcal{X} \tag{23}$$

representing desirable or stable system states.

For example, in a clinical monitoring system, the desirable region may correspond to combinations of respiratory rate, oxygen saturation, and blood pressure within physiologically stable ranges. In an educational system, desirable regions may correspond to combinations of signals indicating coherent reasoning and evidence alignment.

As the system evolves through successive turns, the sequence of state vectors

$$x_0, x_1, x_2, \ldots \tag{24}$$

forms a trajectory through state space. This trajectory may move toward or away from the desirable region $\mathcal{X}_D$.

Policy rules operate on projected state representations to influence these trajectories. When the system moves toward undesirable regions of state space, policy may trigger actions designed to restore the system to stable conditions.

In dynamical systems terminology, desirable regions may behave as *stable attractors*. Interventions guided by policy act to steer the trajectory of the system back toward these regions when deviations occur.

The ESM therefore provides a framework in which policy does not merely respond to isolated signals but instead shapes trajectories through interpretable state space.

10 Determinism, Transparency, and Auditability

A central design principle of the Emergent State Machine architecture is that its reasoning process remains deterministic and inspectable.

The transformation chain within each turn can be expressed as

$$o_t \to S_t \to x_t \to y_t \to a_t \tag{25}$$

where observations are transformed into signals, signals into a state vector, the state vector into policy-space representation, and policy-space coordinates into an action.

Because each stage of this transformation is explicitly defined, system decisions can be traced through the complete reasoning pathway. Any selected action can therefore be analyzed by examining the signals and observations that produced the corresponding policy-space representation.

This deterministic structure provides several important properties.

10.1 Replayability

Given recorded observations and fixed definitions of signals, projection, and policy, the system will reproduce the same action for a given turn. Interaction traces can therefore be replayed to reproduce previous system behavior.

Replayability enables retrospective analysis of system decisions and allows designers to evaluate alternative policies using historical interaction data.

10.2 Failure Localization

When undesirable outcomes occur, the source of the problem can be localized within the reasoning chain. Errors may originate from signal definitions, projection rules, or policy logic, but each stage remains independently inspectable.

Because each component of the architecture is explicit, debugging and system refinement can proceed through targeted revisions rather than opaque model retraining.

10.3 Auditability

The reasoning chain also supports formal auditing. For any action selected by the system, the full sequence of transformations from observation to policy decision can be reconstructed.

This transparency is particularly important in domains where automated decisions influence consequential outcomes, such as healthcare, education, or industrial operations.

Deterministic reasoning therefore provides a structural basis for system accountability.

11 Instrumented Deterministic Evolution

While the Emergent State Machine produces deterministic decisions within a fixed configuration, real-world systems must evolve over time as domain knowledge improves and operational requirements change. The ESM therefore supports a regime of controlled architectural revision called *Instrumented Deterministic Evolution* (IDE).

Under IDE, system behavior evolves only through explicit revision of versioned architectural artifacts. These artifacts include

- signal definitions
- derived signal computations
- projection operators
- policy rules

Any modification to these components produces a new system configuration version.

Because each turn records the observations, signals, state vectors, projections, and actions associated with a decision, historical interaction traces can be replayed under revised configurations. This enables designers to evaluate how policy revisions or signal changes would have affected past system behavior.

Formally, let v denote a specific configuration consisting of signal definitions, projection operators, and policy rules. The action produced by the system during turn t can then be written as

$$a_t^{(v)} = \pi^{(v)} \left(P^{(v)} x_t^{(v)} \right). \tag{26}$$

Evaluating the same observations under a revised configuration v' yields

$$a_t^{(v')} = \pi^{(v')} \left(P^{(v')} x_t^{(v')} \right). \tag{27}$$

Comparing these outcomes allows system designers to study how architectural revisions affect system behavior.

The key property of IDE is that behavioral change occurs only through explicit modification of versioned artifacts. System evolution therefore remains intentional and inspectable rather than emerging implicitly through hidden parameter drift.

Instrumented Deterministic Evolution allows systems to improve over time while preserving replayability, traceability, and governance over consequential decision logic.

12 Boundaries of the Architecture

The Emergent State Machine enforces structural separation between observation, interpretation, and authoritative policy selection. While this separation provides important properties of transparency and replayability, it does not eliminate the inherent limits of modeling, measurement, or governance.

12.1 Projection Quality

Projection determines how signals are interpreted as structural state. If signal definitions are incomplete, poorly designed, or biased, deterministic policy will faithfully act on those flawed representations.

The ESM therefore does not guarantee correctness of interpretation. Instead, it guarantees that interpretation remains explicit, inspectable, and revisable.

12.2 Policy Design

Policy encodes the authoritative decision logic of the system. Deterministic policy may still contain flawed thresholds, incorrect escalation logic, or undesirable governance assumptions.

The architecture ensures that such issues remain visible and modifiable. However, transparency alone does not guarantee that policies themselves are well designed.

12.3 Partial Observability and Clarification

Many real-world decision systems operate under partial observability, where the available signals may not uniquely determine the true system state. Formal frameworks such as partially observable Markov decision processes (POMDPs) model this uncertainty by maintaining belief distributions over latent states [Kaelbling et al., 1998].

The Emergent State Machine adopts a different architectural strategy. Rather than collapsing uncertainty through probabilistic belief updates within the decision mechanism, ambiguity is surfaced explicitly as a system condition. When available signals do not authorize a unique action under policy constraints, the system may request clarification or additional observation before proceeding.

$$\pi(y_t) = \text{request_clarification} \tag{28}$$

Clarification augments the observation set available to the next turn, allowing the system to refine its state representation before selecting a consequential action.

Ambiguity is therefore treated as an observable system condition rather than an internal inference artifact.

12.4 Generative Assistance

Generative models may assist with tasks such as interpretation support, explanation generation, or interface construction. Within the ESM architecture, such models operate at the descriptive layer rather than the authoritative layer.

Generative outputs may contribute to observation processing or signal derivation, but they cannot directly authorize actions. All consequential decisions must pass through deterministic policy evaluation.

This constraint allows generative capabilities to support system reasoning while preserving explicit governance over decision authority.

12.5 Applicability Scope

The Emergent State Machine is most applicable in systems where automated reasoning influences externally consequential decisions. In domains where outputs are purely generative—such as creative text production—the strict separation between interpretation and authority may be unnecessary.

The architecture therefore represents a design discipline for systems in which interpretability, accountability, and governance of automated decisions are critical requirements.

13 Related Work

The Emergent State Machine (ESM) draws on ideas from several established traditions, including control theory, dynamical systems, software architecture, situation awareness research, and recent work on AI interpretability and governance. While each of these areas contributes elements relevant to the architecture, the ESM combines them in a distinct way by enforcing a structural separation between measurement, interpretation, and authoritative policy selection.

13.1 Control and State-Space Systems

Control theory has long represented system behavior using explicit state vectors and state-space models [Kalman, 1960, Åström and Murray, 2010]. Classical formulations distinguish between

measurements, state estimation, and control input, while decision-theoretic control frameworks treat action selection as an explicit policy over system state [Bertsekas, 2017]. Observers estimate system state from measurements, while controllers determine actions based on the estimated state.

The ESM shares this structural intuition. Signals correspond to measured quantities, the state vector provides an explicit representation of the system's condition, and policy governs action selection. However, modern decision systems—particularly those incorporating machine learning—often collapse estimation and control into a single opaque model. The ESM reinstates an explicit architectural boundary between descriptive state representation and authoritative policy selection.

13.2 Dynamical Systems and Trajectories

The representation of system evolution as trajectories through state space is central to dynamical systems theory. Concepts such as attractors, stability regions, and phase-space trajectories provide a mathematical framework for understanding how systems evolve over time and how interventions influence those trajectories [Strogatz, 2015].

The ESM adopts this perspective by treating successive turns as a trajectory through interpretable state space. Policy actions act to steer these trajectories toward desirable regions and away from unstable configurations.

13.3 Situation Awareness

Research in human factors has emphasized the importance of situation awareness in complex operational environments. Situation awareness describes the process by which operators perceive relevant signals, interpret their meaning, and project the implications for system behavior [Endsley, 1995].

The ESM architecture provides a computational analogue to this process. Observations become signals, signals are assembled into a structured state representation, and policy evaluates that state to determine appropriate action. In this sense, the architecture can be viewed as a formalization of situational reasoning within a deterministic computational framework.

13.4 Software Architecture and Separation of Concerns

The idea that complex systems benefit from explicit architectural separation has deep roots in systems science and software engineering. Simon's analysis of hierarchical systems showed that complex systems become manageable when organized into modular structures with clearly defined responsibilities [Simon, 1962]. Later work in software architecture extended these ideas through principles such as modular decomposition and separation of concerns [Parnas, 1972, Dijkstra, 1974]. Architectural patterns such as Model–View–Controller further demonstrate how

separating representation from control logic can reduce coupling between system components [Reenskaug, 1979]. The ESM extends this architectural philosophy to automated reasoning systems. Measurement, interpretation, and decision authority are separated into explicit stages, preventing probabilistic interpretation mechanisms from directly authorizing consequential actions.

13.5 AI Interpretability and Governance

Recent work in AI interpretability and governance has highlighted the risks associated with opaque decision systems [Doshi-Velez and Kim, 2017, Mittelstadt et al., 2016]. Many modern architectures embed both representation and decision logic within learned model parameters, making it difficult to trace decisions to specific inputs or policy assumptions.

The ESM addresses this challenge by ensuring that the reasoning chain from observation to action remains explicit. Signals, projections, and policies are defined as inspectable artifacts whose behavior can be examined and revised independently. This structural transparency enables replayability, failure localization, and controlled system evolution under Instrumented Deterministic Evolution.

13.6 Distinguishing Features of the ESM

While each of the above traditions contributes ideas relevant to the architecture, the Emergent State Machine introduces a distinctive combination of principles:

- reasoning is organized into explicit **turns**
- system state is represented through **interpretable signals and state vectors**
- projection expresses state in **policy-relevant coordinates**
- authoritative action selection remains **deterministic and inspectable**
- system evolution occurs through **Instrumented Deterministic Evolution**

Together, these elements produce an architecture in which automated reasoning remains transparent, reproducible, and governable even as systems evolve over time.

14 Conclusion

This paper has presented the mathematical foundations of the Emergent State Machine (ESM), a deterministic architecture for interpretable situational reasoning in complex systems. The architecture organizes reasoning into discrete units called turns, each of which transforms

observations into signals, constructs an explicit state vector, projects that state into policy-relevant coordinates, and deterministically selects an action.

By structuring system behavior as a sequence of turns, the ESM makes reasoning processes inspectable, replayable, and auditable. State evolution becomes visible as trajectories through interpretable state space, allowing policies to guide the system toward desirable regions and away from unstable conditions.

The architecture preserves a structural boundary between interpretation and authority. Projection mechanisms describe the current situation, while deterministic policy governs consequential action. This separation ensures that system behavior remains traceable to explicit architectural artifacts rather than hidden model parameters.

Because signals, projections, and policies are explicitly defined and versioned, the architecture supports a regime of Instrumented Deterministic Evolution in which behavioral change occurs only through intentional revision of system components. System evolution therefore becomes controlled and inspectable rather than implicit.

While the architecture does not eliminate the challenges of modeling complex domains, it provides a disciplined framework in which automated reasoning remains transparent, reproducible, and governable. In systems where probabilistic interpretation influences consequential decisions, maintaining an explicit boundary between description and authority is not merely a design preference but an architectural requirement.

References

Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control.* Athena Scientific, 1995.

Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control, Vol. I.* Athena Scientific, 2017.

Christopher M. Bishop. *Pattern Recognition and Machine Learning.* Springer, 2006.

Edsger W. Dijkstra. On the role of scientific thought. *Selected Writings on Computing: A Personal Perspective*, pages 60–66, 1974.

Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017.

Mica R. Endsley. Toward a theory of situation awareness in dynamic systems. *Human Factors*, 37(1):32–64, 1995.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. *Planning and Acting in Partially Observable Stochastic Domains*, volume 101. 1998.

Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME–Journal of Basic Engineering*, 82:35–45, 1960.

Lennart Ljung. *System Identification: Theory for the User*. Prentice Hall, 1999.

Brent Mittelstadt, Patrick Allo, Mariarosaria Taddeo, Sandra Wachter, and Luciano Floridi. The ethics of algorithms: Mapping the debate. *Big Data & Society*, 3(2), 2016.

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

Srinivasa Raghavan and Sanjeev Kulkarni. *Signal Detection and Estimation*. Springer, 2014.

Trygve Reenskaug. Models–views–controllers. *Xerox PARC Technical Note*, 1979.

Herbert A. Simon. The architecture of complexity. *Proceedings of the American Philosophical Society*, 106(6):467–482, 1962.

Steven H. Strogatz. *Nonlinear Dynamics and Chaos*. Westview Press, 2015.

Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2010.